

WID Specification

Version: 1.0.0

Status: Draft

Overview

WID (Waldiez/SYNAPSE Identifier) is a time-ordered, human-readable, collision-resistant identifier format designed for distributed IoT and agent systems.

Two variants are supported:

1. **WID** — Simple time-sortable ID
2. **HLC-WID** — Hybrid Logical Clock variant with node identifier

Design Goals

1. **Lexicographically sortable** — IDs sort chronologically as strings
2. **Human-readable** — Timestamps are visible, not encoded
3. **Collision-resistant** — Sequence counters + optional random padding
4. **Distributed** — HLC variant supports node tagging for distributed systems
5. **Compact** — Minimal overhead while meeting the above goals

Format

WID (Simple)

WID ::= TIMESTAMP "." SEQ "Z" ["-" PAD]

HLC-WID (Hybrid Logical Clock)

HLC-WID ::= TIMESTAMP "." LC "Z" "-" NODE ["-" PAD]

Components

Component	Format	Required	Description
TIMESTAMP	YYYYMMDDTHHMMSSs or YYYYMMDDTHHMMSSmmm	Yes	UTC timestamp (ISO 8601 basic format, second or millisecond precision)
SEQ / LC	[0-9]{W}	Yes	Zero-padded sequence/logical counter (width W)
NODE	[A-Za-z0-9_]+HLC only		Node identifier (no hyphens, no spaces)

Component	Format	Required	Description
PAD	[0-9a-f]{Z}	No	Random lowercase hex padding for collision defense

Parameters

Parameter	Default	Description
W	4	Sequence/LC width (digits). Supports 10^W IDs per second
Z	6	Padding length (hex chars). 0 disables padding
time_unit	sec	Timestamp precision mode: sec or ms

Examples

WID (W=4, Z=0)

```
20260212T091530.0000Z
20260212T091530.0001Z
20260212T091530.0002Z
```

WID with Padding (W=4, Z=6)

```
20260212T091530.0000Z-a3f91c
20260212T091530.0001Z-7b2e4f
```

HLC-WID (W=4, Z=0)

```
20260212T091530.0000Z-node01
20260212T091530.0001Z-node01
```

HLC-WID with Padding (W=4, Z=6)

```
20260212T091530.0000Z-node01-a3f91c
20260212T091530.0001Z-node01-7b2e4f
```

EBNF Grammar

```
WID      ::= TIMESTAMP "." SEQ "Z" [ "-" PAD ]
HLC_WID  ::= TIMESTAMP "." LC "Z" "-" NODE [ "-" PAD ]

TIMESTAMP ::= TIMESTAMP_SEC | TIMESTAMP_MS
TIMESTAMP_SEC ::= YYYY MM DD "T" HH MI SS
TIMESTAMP_MS  ::= YYYY MM DD "T" HH MI SS mmm
YYYY         ::= DIGIT DIGIT DIGIT DIGIT
MM           ::= DIGIT DIGIT (* 01-12 *)
```

```

DD      ::= DIGIT DIGIT                      (* 01-31 *)
HH      ::= DIGIT DIGIT                      (* 00-23 *)
MI      ::= DIGIT DIGIT                      (* 00-59 *)
SS      ::= DIGIT DIGIT                      (* 00-59 *)
mmm     ::= DIGIT DIGIT DIGIT                (* 000-999 *)

SEQ      ::= DIGIT{W}                        (* fixed width, zero-padded *)
LC       ::= DIGIT{W}                        (* logical counter, zero-padded *)

NODE     ::= ALNUM { ALNUM | "_" }           (* no hyphens allowed *)

PAD      ::= HEX{Z}                          (* lowercase random hex *)

DIGIT    ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
HEX      ::= DIGIT | "a" | "b" | "c" | "d" | "e" | "f"
ALNUM    ::= DIGIT | LETTER
LETTER   ::= "a"..."z" | "A"..."Z"

```

Generation Algorithm

WID Generator

```

function next_wid(W, Z, time_unit):
    now_tick = current_utc_tick(time_unit)    # sec or ms

    # Ensure monotonicity
    if now_tick <= last_tick:
        tick = last_tick
    else:
        tick = now_tick

    # Increment sequence
    if tick == last_tick:
        seq = last_seq + 1
    else:
        seq = 0

    # Handle sequence overflow
    if seq > 10W - 1:
        tick = tick + 1
        seq = 0

    # Update state
    last_tick = tick
    last_seq = seq

```

```

# Format components
ts = format_timestamp(tick, time_unit) # "YYYYMMDDTHHMMSS" | "YYYYMMDDTHHMMSSmmm"
seq_str = zero_pad(seq, W)           # W digits

# Build WID
if Z > 0:
    pad = random_hex(Z)
    return ts + "." + seq_str + "Z-" + pad
else:
    return ts + "." + seq_str + "Z"

```

HLC-WID Generator

```

function next_hlc_wid(W, Z, node, time_unit):
    now = current_utc_tick(time_unit) # sec or ms

    # Update physical time
    if now > pt:
        pt = now
        lc = 0
    else:
        lc = lc + 1

    # Handle overflow
    if lc > 10^W - 1:
        pt = pt + 1
        lc = 0

    # Format
    ts = format_timestamp(pt, time_unit)
    lc_str = zero_pad(lc, W)

    if Z > 0:
        pad = random_hex(Z)
        return ts + "." + lc_str + "Z-" + node + "-" + pad
    else:
        return ts + "." + lc_str + "Z-" + node

function observe(remote_pt, remote_lc):
    now = current_utc_tick(time_unit)
    new_pt = max(now, pt, remote_pt)

    if new_pt == pt == remote_pt:
        lc = max(lc, remote_lc) + 1
    elif new_pt == pt:
        lc = lc + 1

```

```

elif new_pt == remote_pt:
    lc = remote_lc + 1
else:
    lc = 0

pt = new_pt
# Handle overflow
if lc > 10^W - 1:
    pt = pt + 1
    lc = 0

```

Validation Regex

WID (W=4, Z=0) Regex

$^{[0-9]\{8\}}T[0-9]\{6\}\backslash.[0-9]\{4\}Z\$$

WID (W=4, Z=6) Regex

$^{[0-9]\{8\}}T[0-9]\{6\}\backslash.[0-9]\{4\}Z(?:-[0-9a-f]\{6\})? \$$

WID ms (W=4, Z=0) Regex

$^{[0-9]\{8\}}T[0-9]\{9\}\backslash.[0-9]\{4\}Z\$$

HLC-WID (W=4, Z=0) Regex

$^{[0-9]\{8\}}T[0-9]\{6\}\backslash.[0-9]\{4\}Z-[A-Za-z0-9_]+\$$

HLC-WID (W=4, Z=6) Regex

$^{[0-9]\{8\}}T[0-9]\{6\}\backslash.[0-9]\{4\}Z-[A-Za-z0-9_]+(?:-[0-9a-f]\{6\})? \$$

HLC-WID ms (W=4, Z=6) Regex

$^{[0-9]\{8\}}T[0-9]\{9\}\backslash.[0-9]\{4\}Z-[A-Za-z0-9_]+(?:-[0-9a-f]\{6\})? \$$

Ordering Properties

WIDs are designed to be **lexicographically sortable**:

1. Timestamp sorts first (most significant)
2. Sequence/LC number sorts within the same second
3. Node (HLC only) provides tie-breaking for distributed systems
4. Padding does not affect logical ordering

Note: When $Z > 0$, strict monotonicity is not guaranteed due to random padding, but chronological ordering is preserved by timestamp+sequence.

Key Differences: WID vs HLC-WID

Aspect	WID	HLC-WID
Use case	Single-node, local	Distributed, multi-node
Node tag	None	Required
Clock model	Physical time only	Hybrid Logical Clock
Merge support	No	Yes (<code>observe</code> function)
Example	<code>...0042Z-a3f91c</code>	<code>...0042Z-node01-a3f91c</code>

Conformance

Implementations **MUST** pass all test cases in `conformance/` to be considered compliant.

See:

- `conformance/valid.json` — WIDs that **MUST** be accepted
- `conformance/invalid.json` — WIDs that **MUST** be rejected
- `conformance/generation.json` — Generation behavior tests

WID Millisecond Mode (**W=4**, **Z=0**, `time_unit=ms`)

`20260212T091530123.0000Z`

`20260212T091530123.0001Z`